DIA 13

Segurança, Autorização e Multi-Tenancy

Controle de acesso no backend, isolamento por clínica e rastreabilidade

Treinamento prático com decisões técnicas, código executável e critérios de qualidade

Roteiro da aula

01	**Modelo de ameaça**	Identidade, permissão e fronteira do tenant
02	**Autorização**	Policies, Gates e permissões aplicadas no backend
03	**Isolamento e proteção**	Queries, APIs, rate limit e auditoria
04	**Projeto do dia**	Provar que uma clínica não acessa dados de outra

Risco real no ClinicFlow

Um recepcionista altera o ID da rota e recebe um agendamento de outra clínica. O frontend ocultava o botão, mas a API não validava o tenant.

FALHAS

- IDOR entre clínicas
- permissão somente na interface
- logs com dados sensíveis

CONTROLES

- autorização no backend
- tenant derivado da sessão
- teste negativo e auditoria

Autenticação e autorização

Autenticação

Confirma a identidade por sessão, token ou outro mecanismo confiável.

- quem é o usuário
- token ainda é válido
- qual guard autenticou

Autorização

Decide se a identidade pode executar uma ação sobre um recurso específico.

- qual capacidade
- qual recurso
- qual tenant

Estar autenticado não concede acesso automático ao recurso solicitado.

Policy com tenant e capacidade

A Policy centraliza a decisão sobre a entidade e considera a clínica antes da permissão funcional.

```
public function update(User $user, Appointment $appointment): bool
{
    return $user->clinic_id === $appointment->clinic_id
        && $user->can('appointments.update');
}

$this->authorize('update', $appointment);
```

Teste obrigatório

- mesma clínica e permitido
- mesma clínica e negado
- outra clínica e negado
- recurso inexistente

A mesma Policy deve proteger controller, API e qualquer outro ponto de entrada.

Gate para capacidade sem model

Gates atendem decisões globais. Permissões continuam explícitas e independentes do nome da role.

```
Gate::define('view-financial-dashboard', function (User $user) {
    return $user->clinic_id !== null
        && $user->can('reports.financial.view');
});

Gate::authorize('view-financial-dashboard');
```

Roles

- agrupam permissões
- não substituem Policies
- mudam por contexto
- devem ter privilégio mínimo

Autorize capacidades. Evite espalhar verificações de role por controllers e views.

Tenant derivado do usuário autenticado

O cliente não escolhe a clínica por parâmetro. Um contexto confiável resolve o tenant para a requisição atual.

```
final class ResolveTenant
{
    public function handle(Request $request, Closure $next)
    {
        $clinic = $request->user()?->clinic;
        abort_unless($clinic, 403);

        app(CurrentClinic::class)->set($clinic);

        return $next($request);
    }
}
```

Fronteira

- subdomínio validado
- sessão vinculada
- contexto por request
- falha fechada sem tenant

Jobs e comandos precisam receber clinic_id explicitamente porque não compartilham a requisição HTTP.

Consultas isoladas por clínica

A query começa pelo tenant atual e só depois aplica identificadores fornecidos pelo cliente.

```
$appointment = Appointment::query()
    ->whereBelongsTo(app(CurrentClinic::class)->get())
    ->whereKey($request->route('appointment'))
    ->firstOrFail();

$this->authorize('view', $appointment);
```

Defesa em camadas

- filtro de tenant
- Policy
- índices compostos
- testes de acesso cruzado

Um global scope reduz omissões, mas exige regras claras para tarefas administrativas e jobs.

API com Sanctum e Resource

Autentique o token, aplique autorização e exponha somente campos necessários ao consumidor.

```
Route::middleware([
    'auth:sanctum',
    'abilities:appointments.view',
    ResolveTenant::class,
])->get('/appointments/{appointment}', ShowAppointmentController::class);

public function __invoke(Appointment $appointment)
{
    $this->authorize('view', $appointment);

    return AppointmentResource::make($appointment);
}
```

Contrato

- escopo do token
- Policy executada
- campos mínimos
- erros sem detalhes internos

Resource controla serialização. Ele não substitui autorização nem isolamento do tenant.

Rate limit por usuário e clínica

A chave deve representar quem consome e qual recurso está protegido. Limites globais podem prejudicar tenants legítimos.

```
RateLimiter::for('appointments', function (Request $request) {
    $user = $request->user();

    return Limit::perMinute(120)
        ->by($user->clinic_id.':'.$user->id);
});
```

Resposta

- HTTP 429
- Retry-After
- métrica de bloqueios
- limite diferente por operação

Rate limiting reduz abuso, mas não corrige queries caras nem substitui controle de acesso.

Auditoria de ações críticas

Registre o ator, tenant, entidade e mudança sem copiar dados clínicos ou credenciais.

```
AuditEntry::create([
    'actor_id' => $user->id,
    'clinic_id' => $user->clinic_id,
    'action' => 'appointment.cancelled',
    'subject_type' => Appointment::class,
    'subject_id' => $appointment->id,
    'metadata' => ['reason_code' => $reason->value],
]);
```

Requisitos

- imutabilidade
- retenção definida
- acesso restrito
- horário confiável

Auditoria prova quem fez o quê. Logs operacionais explicam como o sistema se comportou.

Teste contra acesso cruzado

O teste negativo demonstra o isolamento. Use entidades reais de dois tenants e confirme que nada foi alterado.

```
public function test_blocks_cross_tenant_update(): void
{
    $user = User::factory()->for(Clinic::factory())->create();
    $appointment = Appointment::factory()->for(Clinic::factory())
        ->create(['status' => 'scheduled']);
    $this->actingAs($user)->patchJson(
        "/api/appointments/{$appointment->id}",
        ['status' => 'cancelled']
    )->assertNotFound();
    $this->assertSame('scheduled', $appointment->fresh()->status);
}
```

Cobertura

- leitura
- alteração
- exportação
- job assíncrono

Responder 404 pode reduzir enumeração de recursos quando o consumidor não deve conhecer sua existência.

Checklist de segurança aplicado

Priorize controles verificáveis no fluxo real da aplicação.

Acesso

- autorização no backend
- privilégio mínimo
- tenant em toda consulta

Dados

- validação de entrada
- serialização mínima
- segredos fora do repositório

Operação

- dependências atualizadas
- rate limit
- auditoria e alertas

A revisão deve buscar caminhos alternativos: API, job, exportação, comando e painel administrativo.

Projeto do dia: fronteira entre clínicas

Missão

Proteger agendamentos e relatórios para impedir acesso cruzado entre tenants.

Entregas

- CurrentClinic por requisição
- Policies e Gate aplicados
- API com Resource e rate limit
- auditoria de cancelamento

Aceite

- teste entre duas clínicas
- nenhuma autorização só no frontend
- job recebe clinic_id
- logs sem dados sensíveis

A apresentação da equipe deve ligar problema, decisão, evidência e resultado.

Checklist de autorização

1	**Identificar**	usuário e tenant por fonte confiável
2	**Restringir**	query começa pelo tenant atual
3	**Autorizar**	Policy ou Gate no backend
4	**Minimizar**	retorno contém somente campos necessários
5	**Provar**	teste negativo e auditoria

Saída esperada: acesso permitido por capacidade e recurso, sem vazamento entre clínicas.